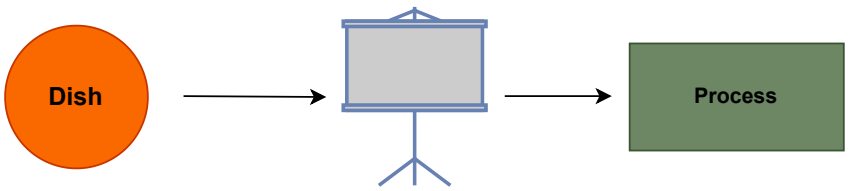
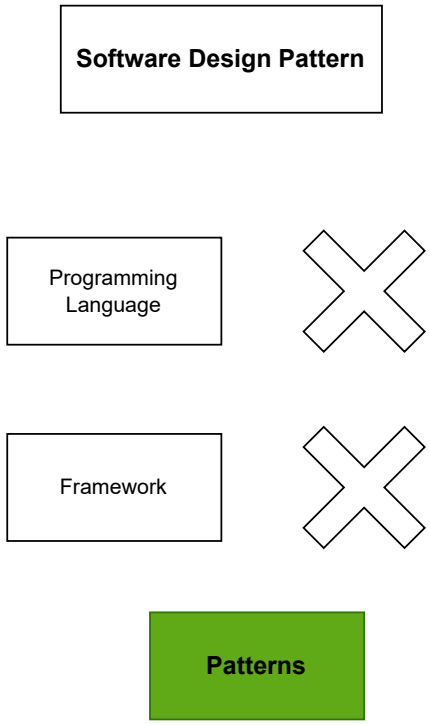
Software Design Patterns

Why?


Dish
Process

Why Use Them?

- Reusability
- Cleaner Code
- Easier Maintenance
- Scalability
- Shared Vocabulary
- Better Collaboration


Software Design Pattern
Programming Language
Framework
Patterns

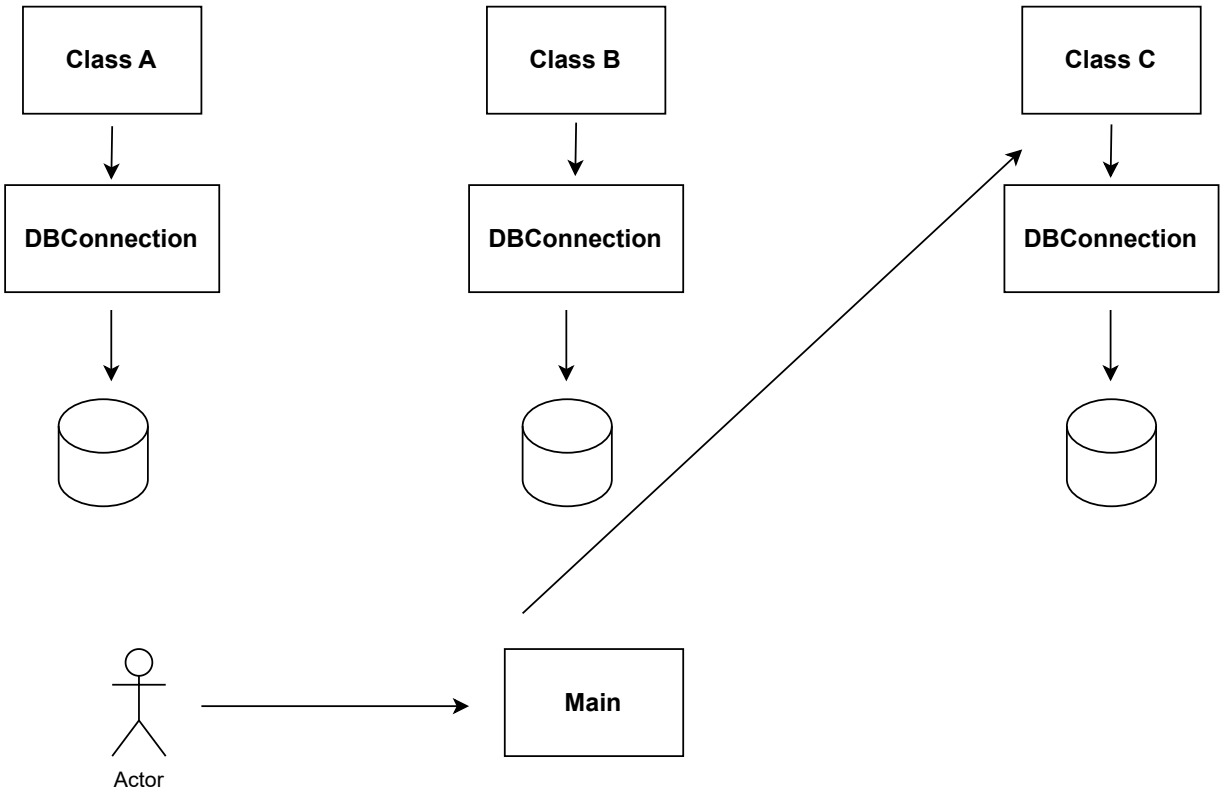
List of Design Patterns

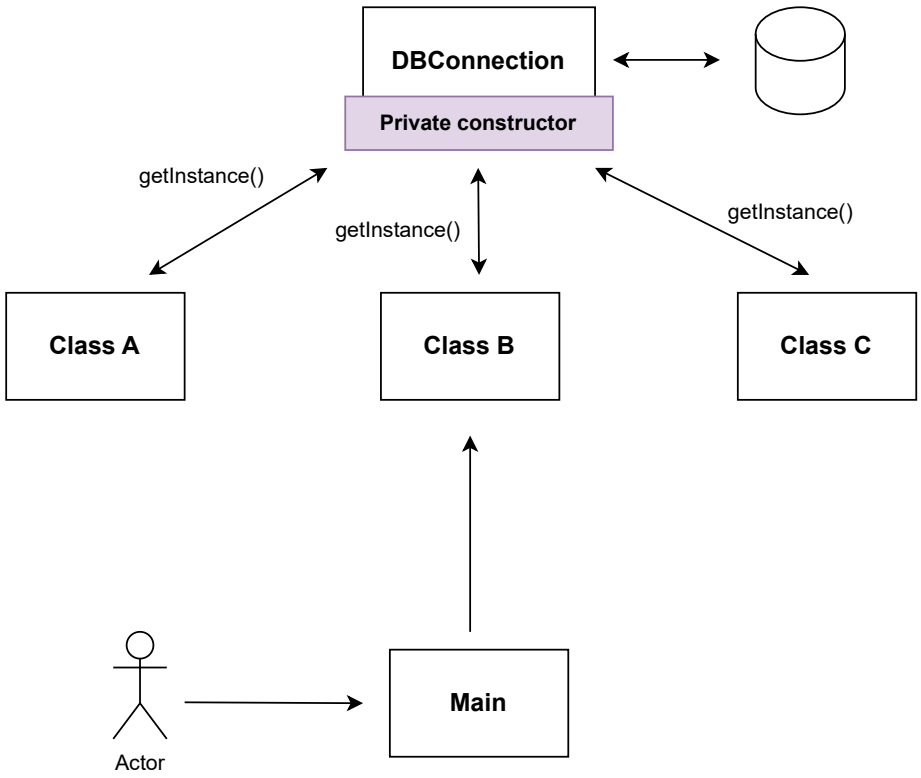
Creational Patterns	Structural Pattern	Behavioral Pattern
Singleton	Adapter	Chain of Responsibility
Factory Method	Bridge	Command
Abstract Factory	Composite	Iterator
Builder	Decorator	Mediator
Prototype	Facade	Observer
	Flyweight	State
	Proxy	Strategy
		Template Method

Singleton Desing Pattern

The Singleton Design Pattern is a way to make sure that a class has only one object ever created from it — and that object can be accessed from anywhere in your code.

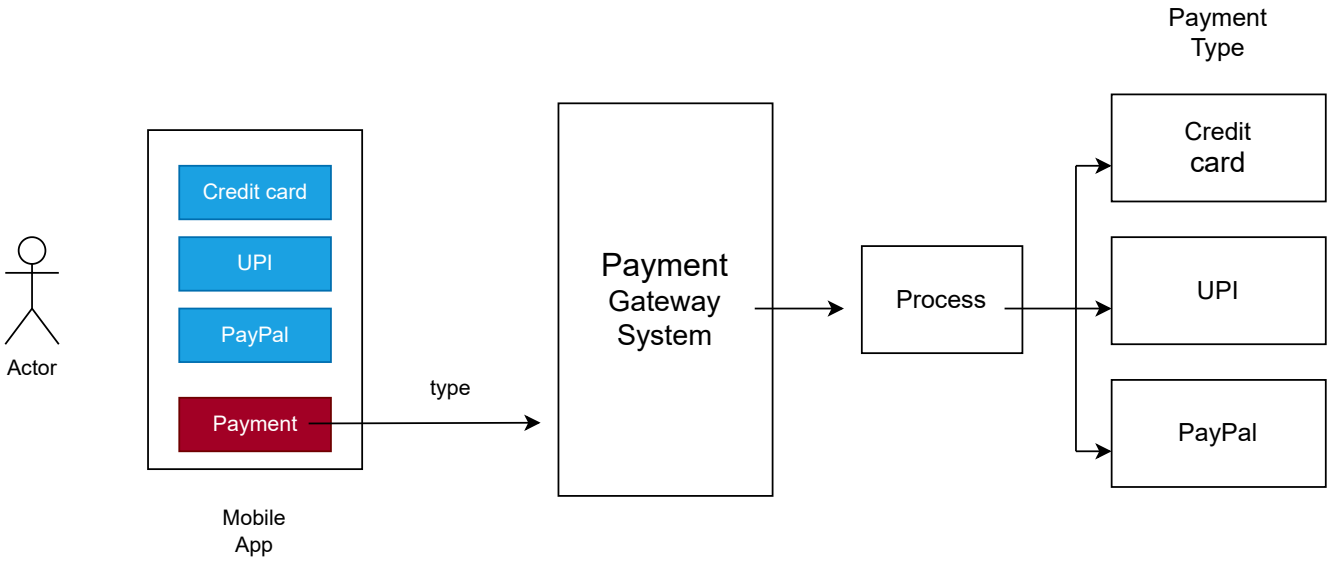
This pattern is often used for things like database connections, logging, or configuration settings — where having more than one instance could cause problems or waste resources.


Class A
Class B
Class C
DBConnection
DBConnection
DBConnection
Main
Actor

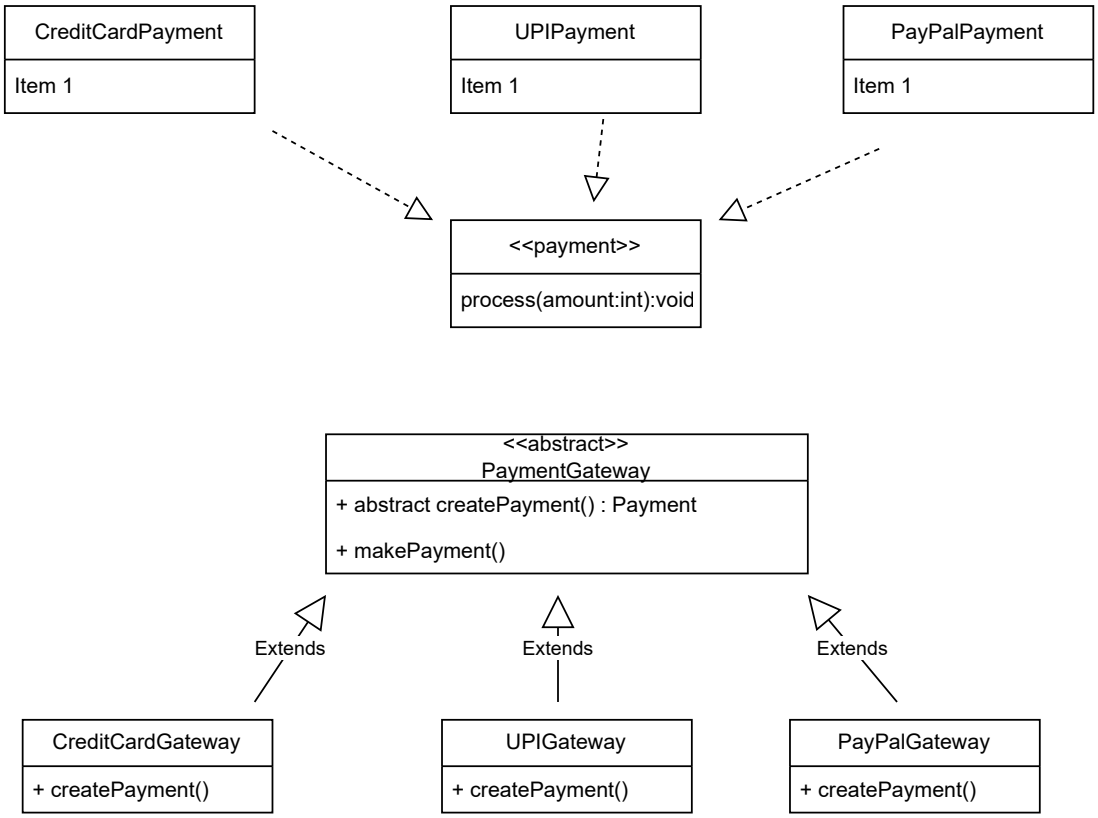

DBConnection
Private constructor
getInstance()
getInstance()
getInstance()
Class A
Class B
Class C
Main
Actor

Factory Method Design Pattern

The Factory Method design pattern is a *creational pattern* that defines an interface for creating an object, but allows subclasses to alter the type of objects that will be created.

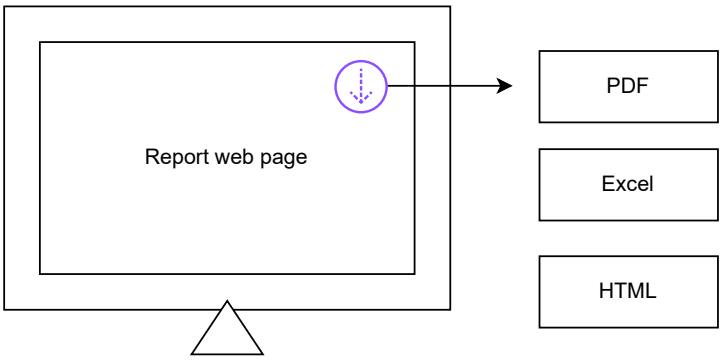

Payment
Type
Credit
card
Credit card
UPI
PayPal
Payment
Actor
type
Payment
Gateway
System
Process
UPI
PayPal
Mobile
App

Class UML Diagram

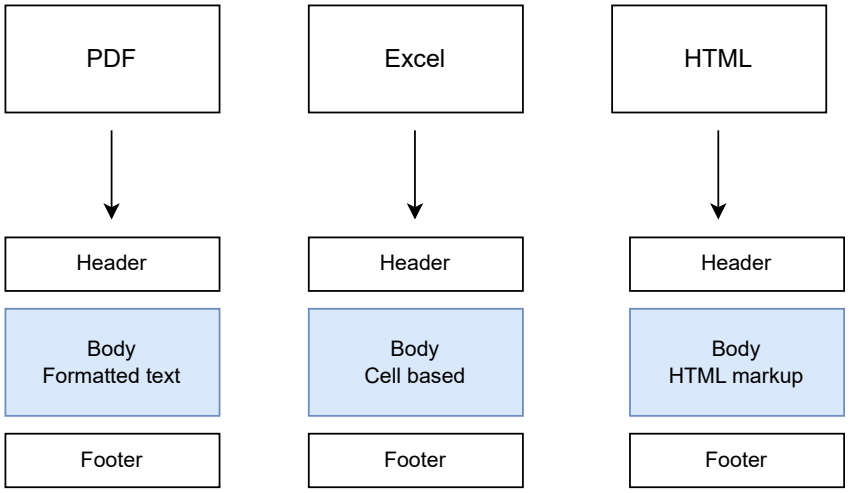


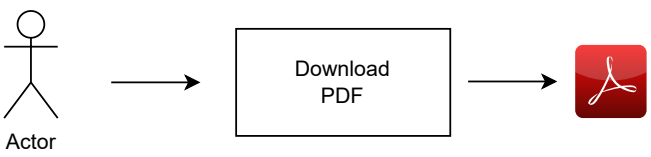
Abstract Factory Design Pattern

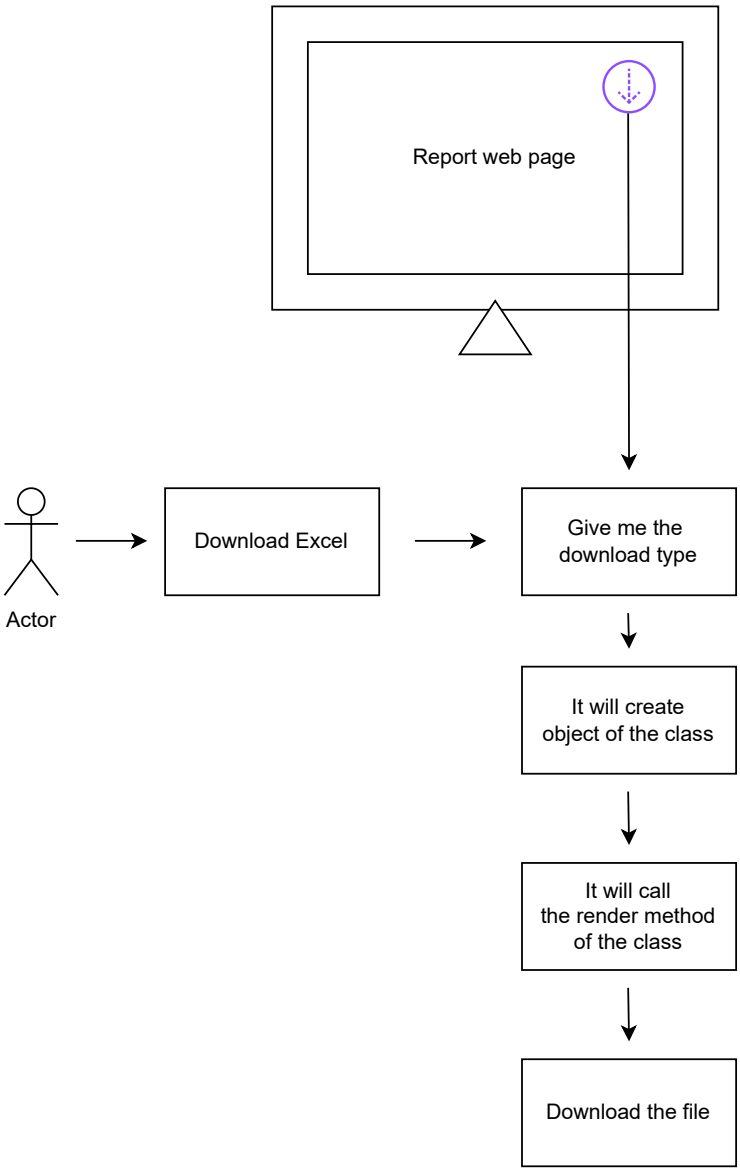
Problem Statement



JSON

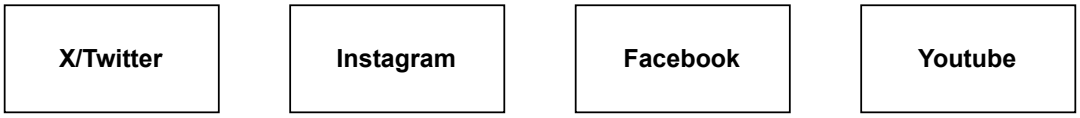


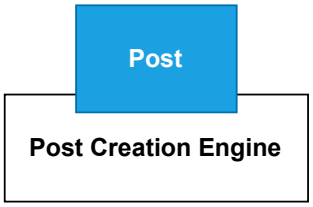


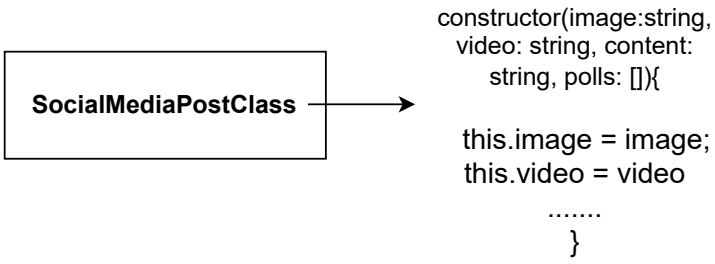

Report web page
Actor
Download Excel
Give me the
download type
It will create
object of the class
It will call
the render method
of the class
Download the file

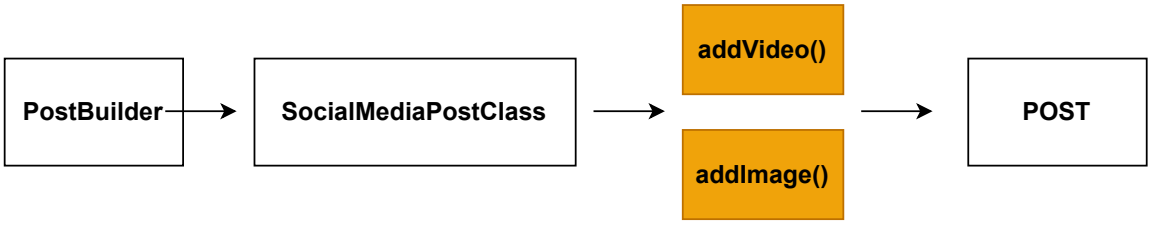
Builder Design Pattern

The Builder pattern lets you construct an object step by step, often using a chain of methods. It’s especially useful when the object has a lot of optional parts or complex setup.


X/Twitter
Instagram
Facebook
Youtube

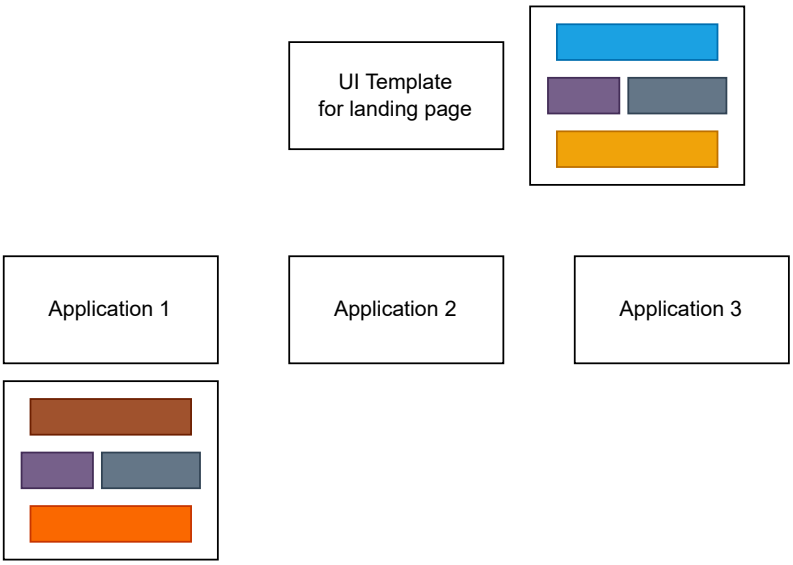

Post
Post Creation Engine


SocialMediaPostClass
constructor(image:string, video: string, content: string, polls: []){
this.image = image;
this.video = video
.....
}


PostBuilder
SocialMediaPostClass
addVideo()
addImage()
POST

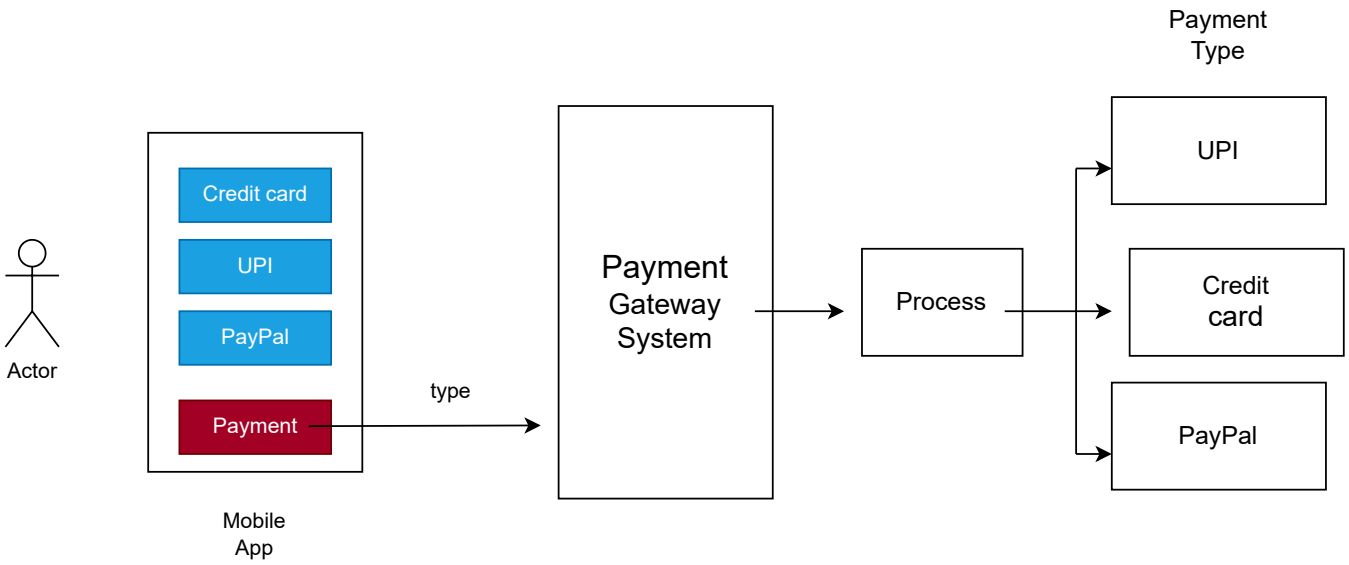
Prototype Design Pattern

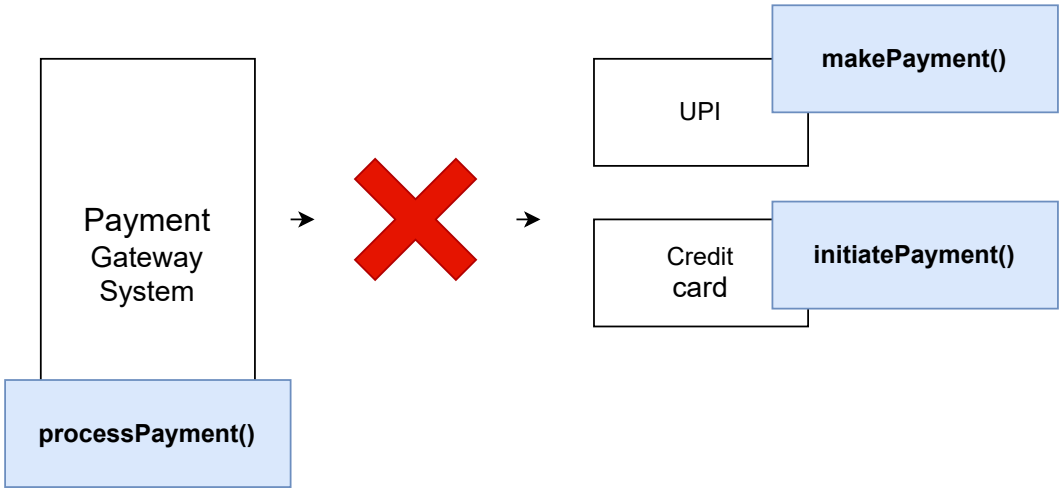
The Prototype Pattern is a creational design pattern that lets you clone existing objects, rather than creating new ones from scratch

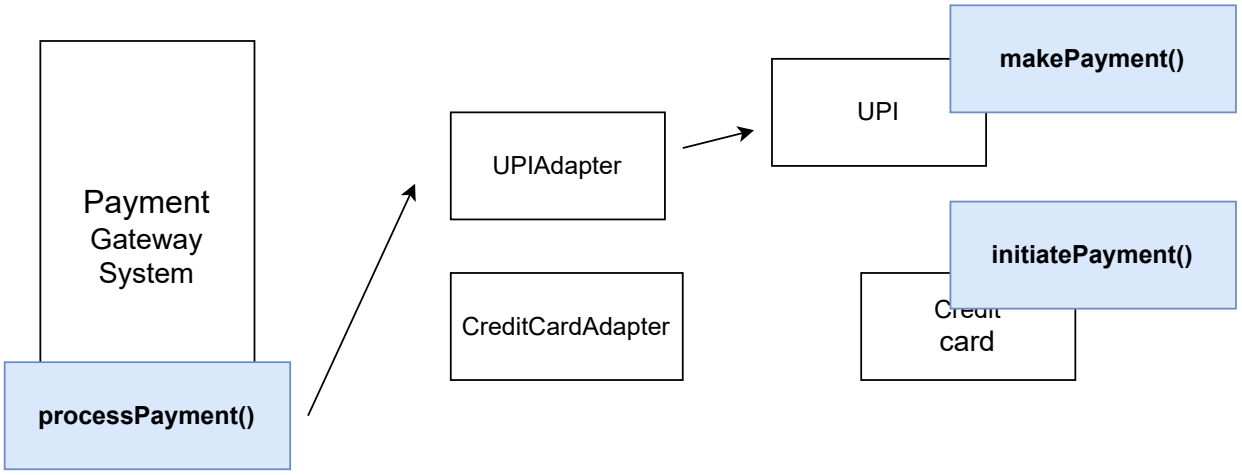

UI Template
for landing page
Application 1
Application 2
Application 3

Adapter Design Pattern

It lets you plug one thing into another that wouldn’t normally fit—without modifying their actual code.

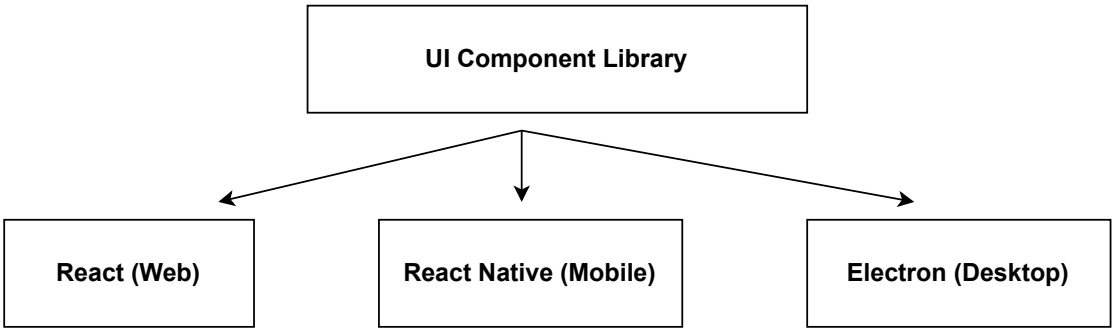

Payment
Type
Actor
Credit card
UPI
PayPal
Payment
type
Mobile
App
Payment
Gateway
System
Process
UPI
Credit
card
PayPal

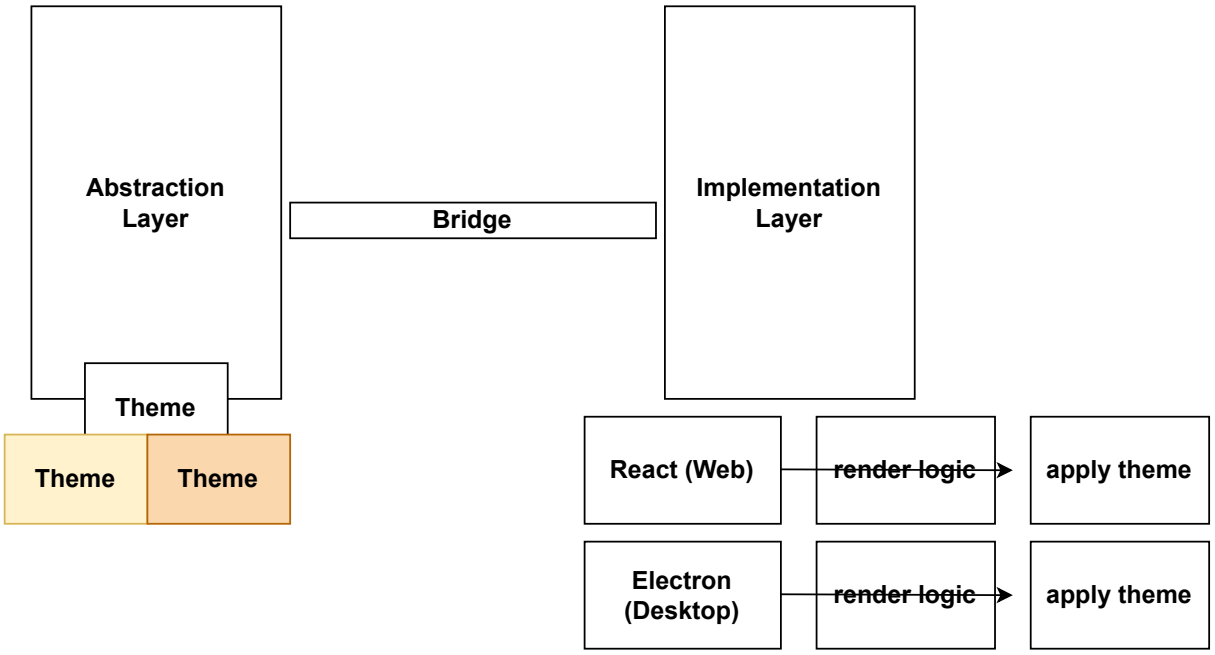

Payment
Gateway
System
processPayment()
UPI
makePayment()
Credit
card
initiatePayment()


Payment
Gateway
System
processPayment()
UPIAdapter
CreditCardAdapter
UPI
makePayment()
initiatePayment()
card

Bridge Design Pattern

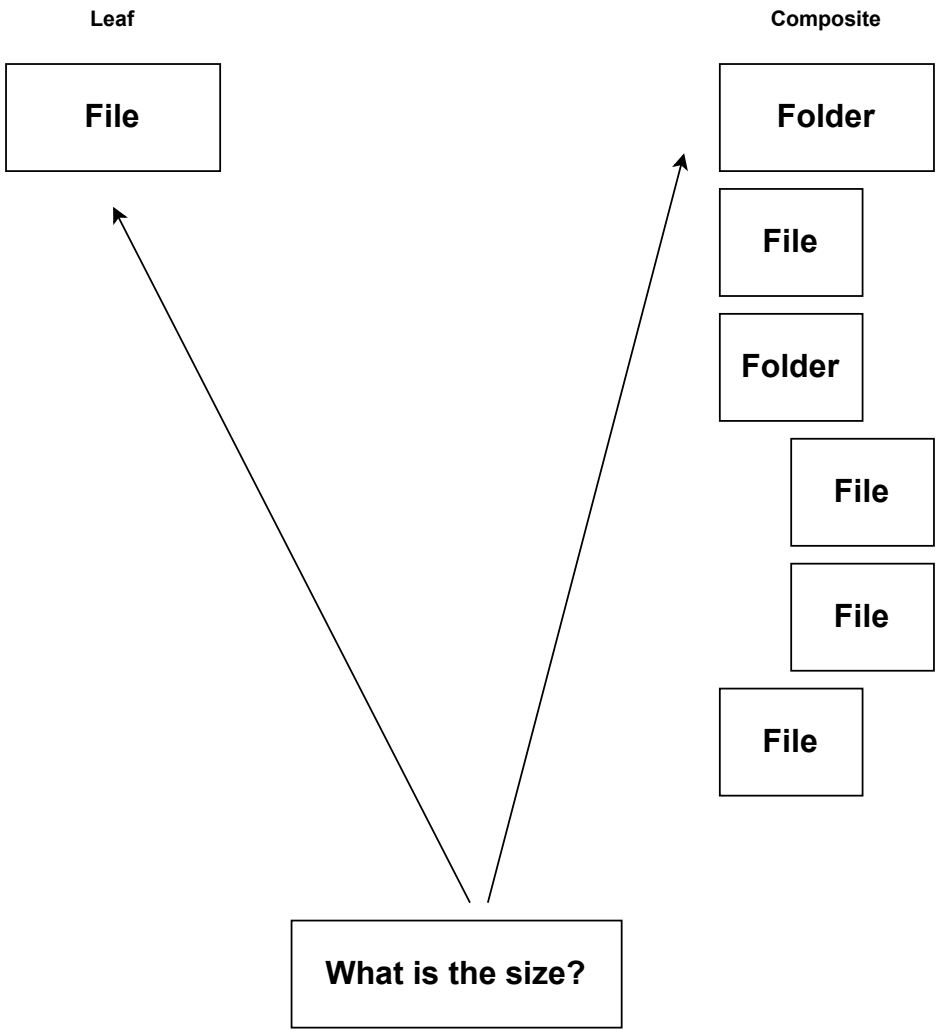
The Bridge pattern separates an abstraction (the high-level logic) from its implementation (low-level details). It’s like creating a bridge between two hierarchies: one for abstractions, and another for implementations, so changes in one don't affect the other.

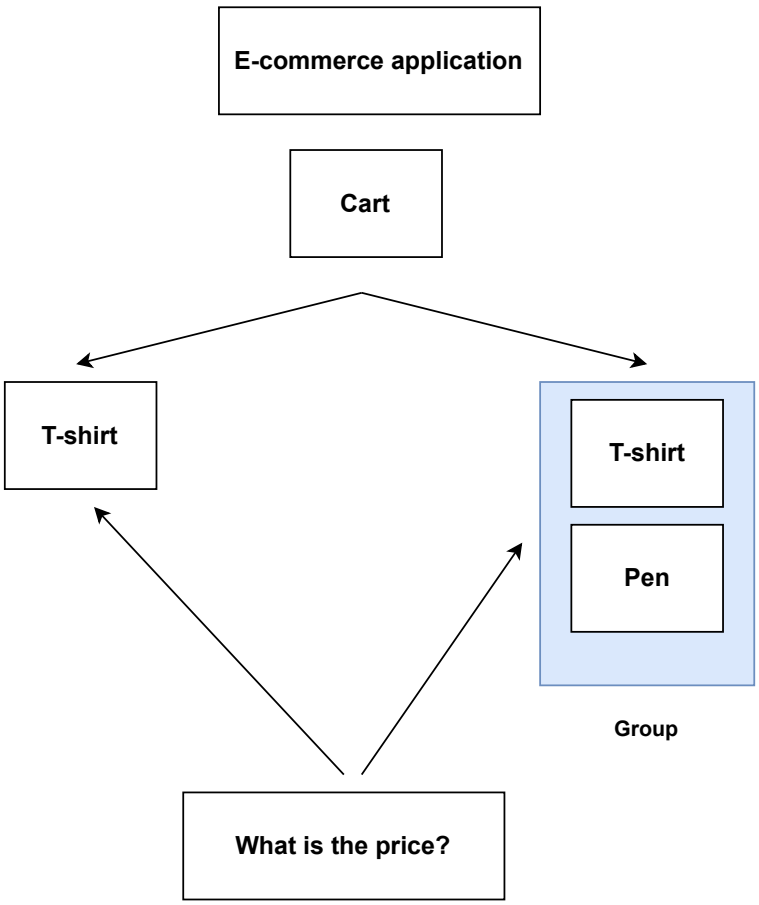

UI Component Library
React (Web)
React Native (Mobile)
Electron (Desktop)


Abstraction Layer
Bridge
Implementation Layer
Theme
Theme
Theme
React (Web)
render logic
apply theme
Electron (Desktop)
render logic
apply theme

Composite Design Pattern

The Composite pattern lets you **treat individual objects and compositions of objects uniformly**. You can model tree-like structures where **leaf nodes** and **composite nodes** share the same interface.


Leaf
Composite
File
Folder
File
Folder
File
File
File
What is the size?

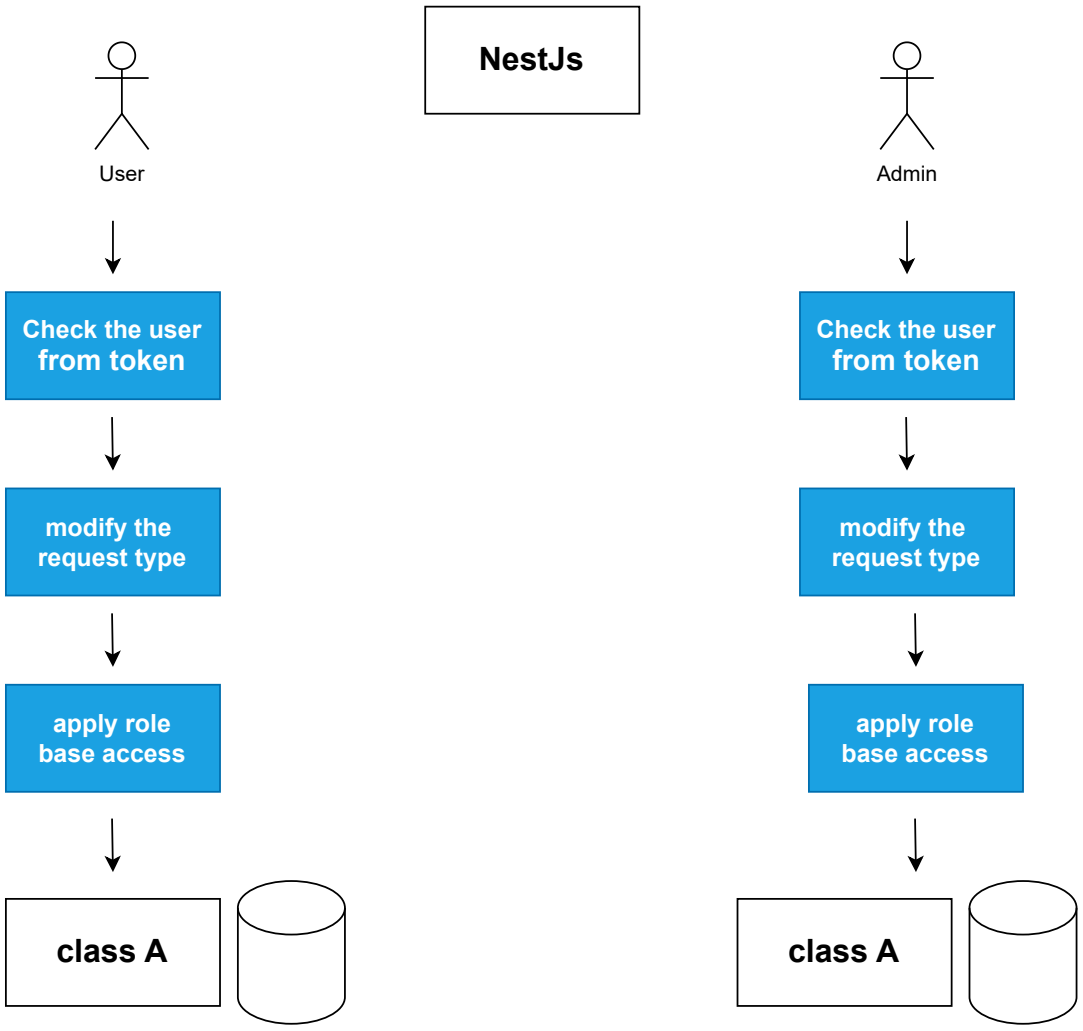

E-commerce application
Cart
T-shirt
T-shirt
Pen
Group
What is the price?

Benefits

- **You can treat a single item or a bundle of items the same way.**
  - **Makes it easy to manage nested product structures.**
  - **Clients (like frontend code) don’t need to care if it’s a product or a bundle —just call `.getPrice()` and `.getName()`.**

Decorator Design Pattern

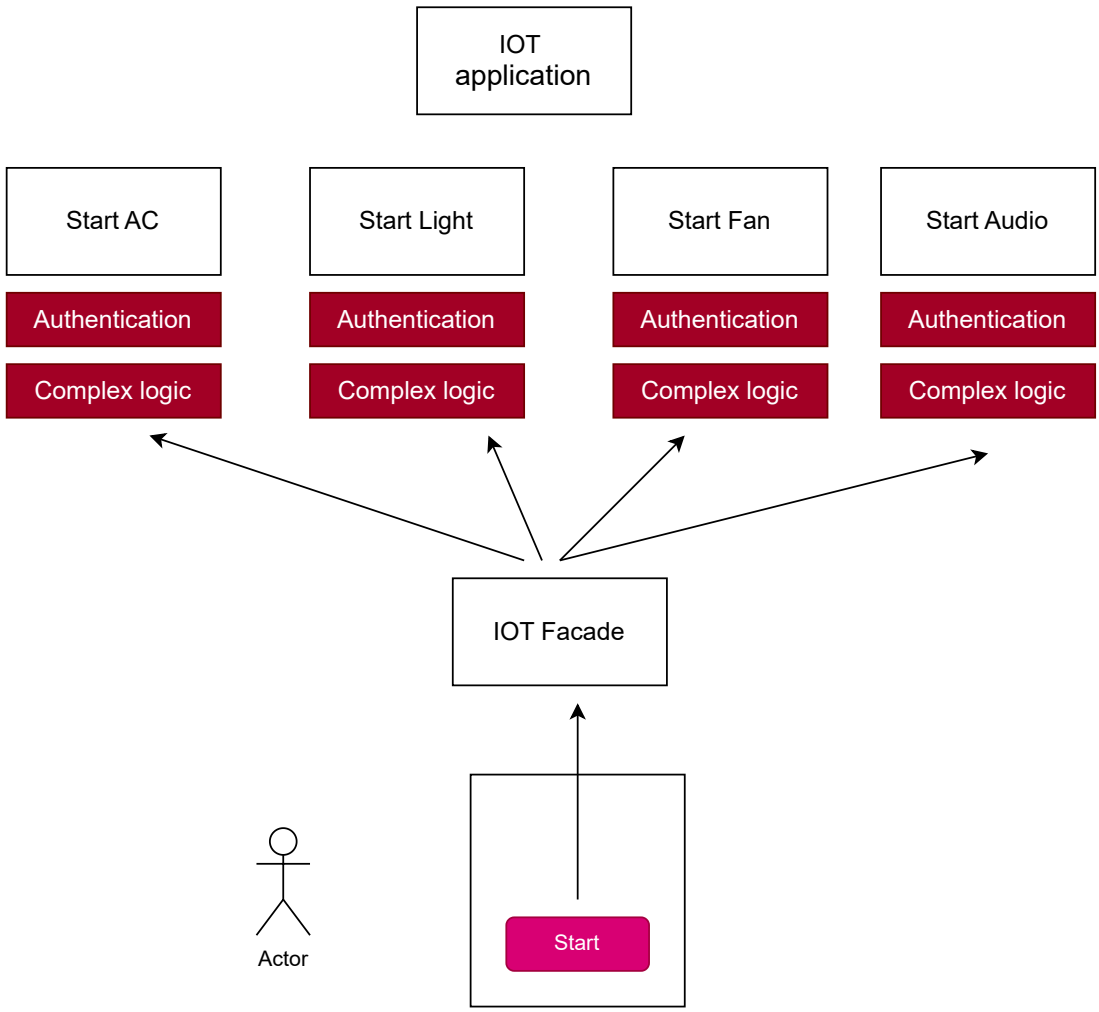
The **Decorator Design Pattern** is a structural pattern that lets you dynamically add behavior or responsibilities to objects without modifying their original code. It’s like wrapping a gift box with layers of decorative paper—each layer adds something new, but the box inside remains unchanged.

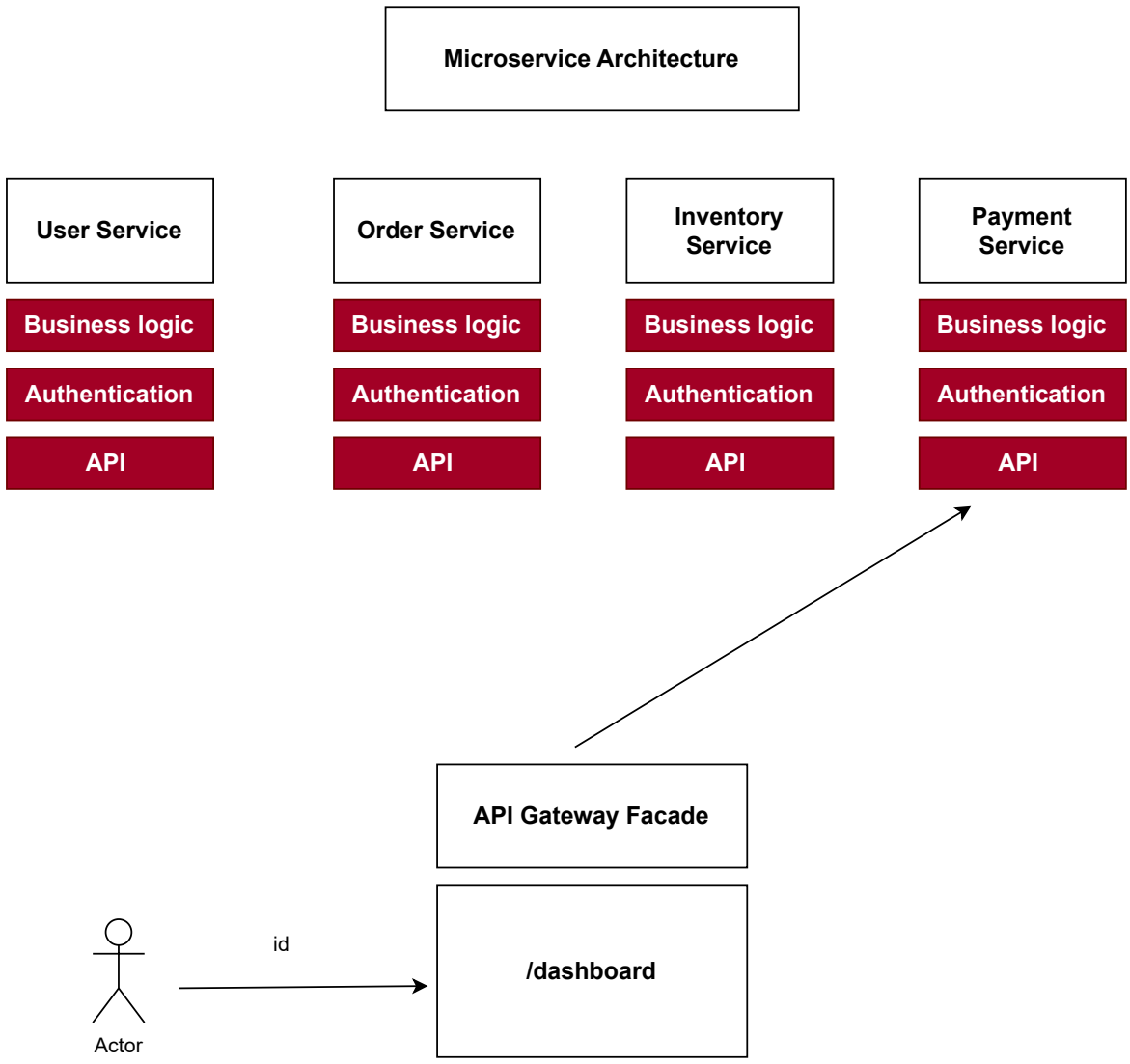

NestJs
User
Admin
Check the user from token
modify the request type
apply role base access
class A
Check the user from token
modify the request type
apply role base access
class A

Facade Design Pattern

The **Facade Design Pattern** is all about providing a simplified interface to a complex subsystem. It's like giving users a clean, easy-to-use remote control for a machine that has tons of internal wiring and components—they don't need to know how it all works, just which buttons to press.

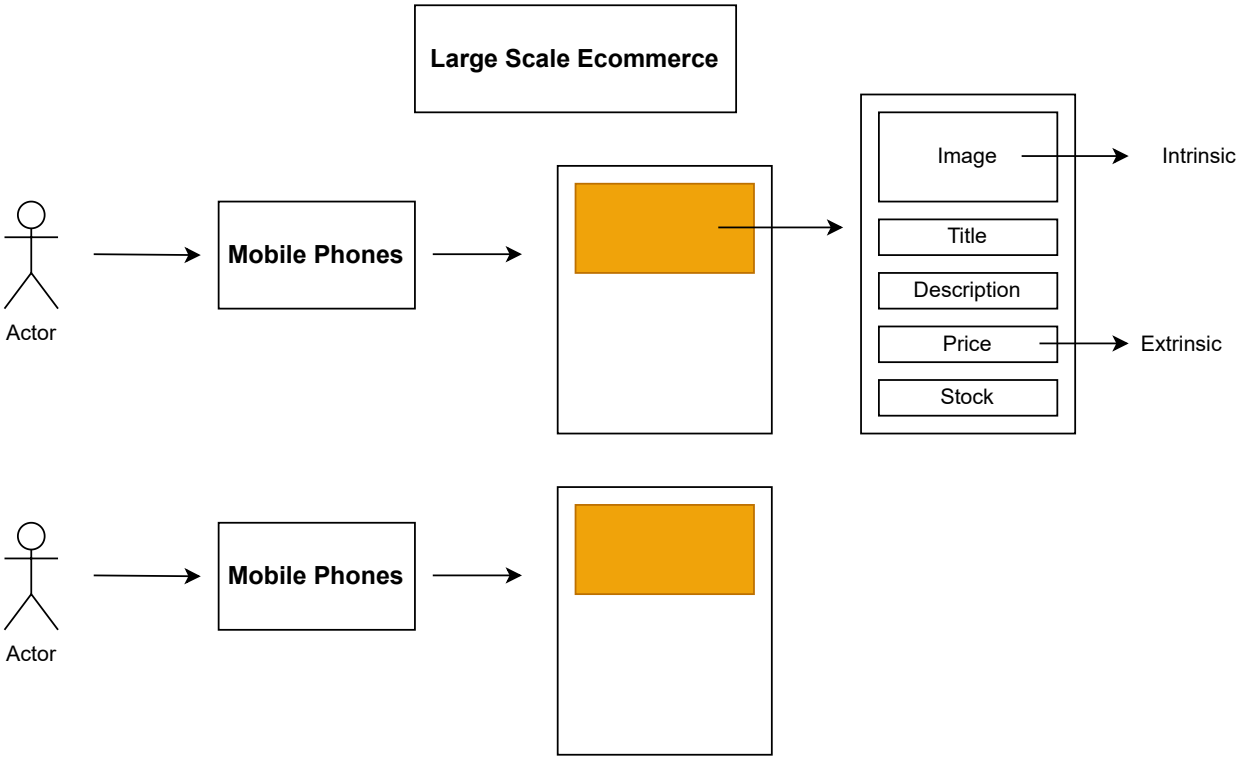
Use case: When you want to simplify interactions with a complex system or group of classes.

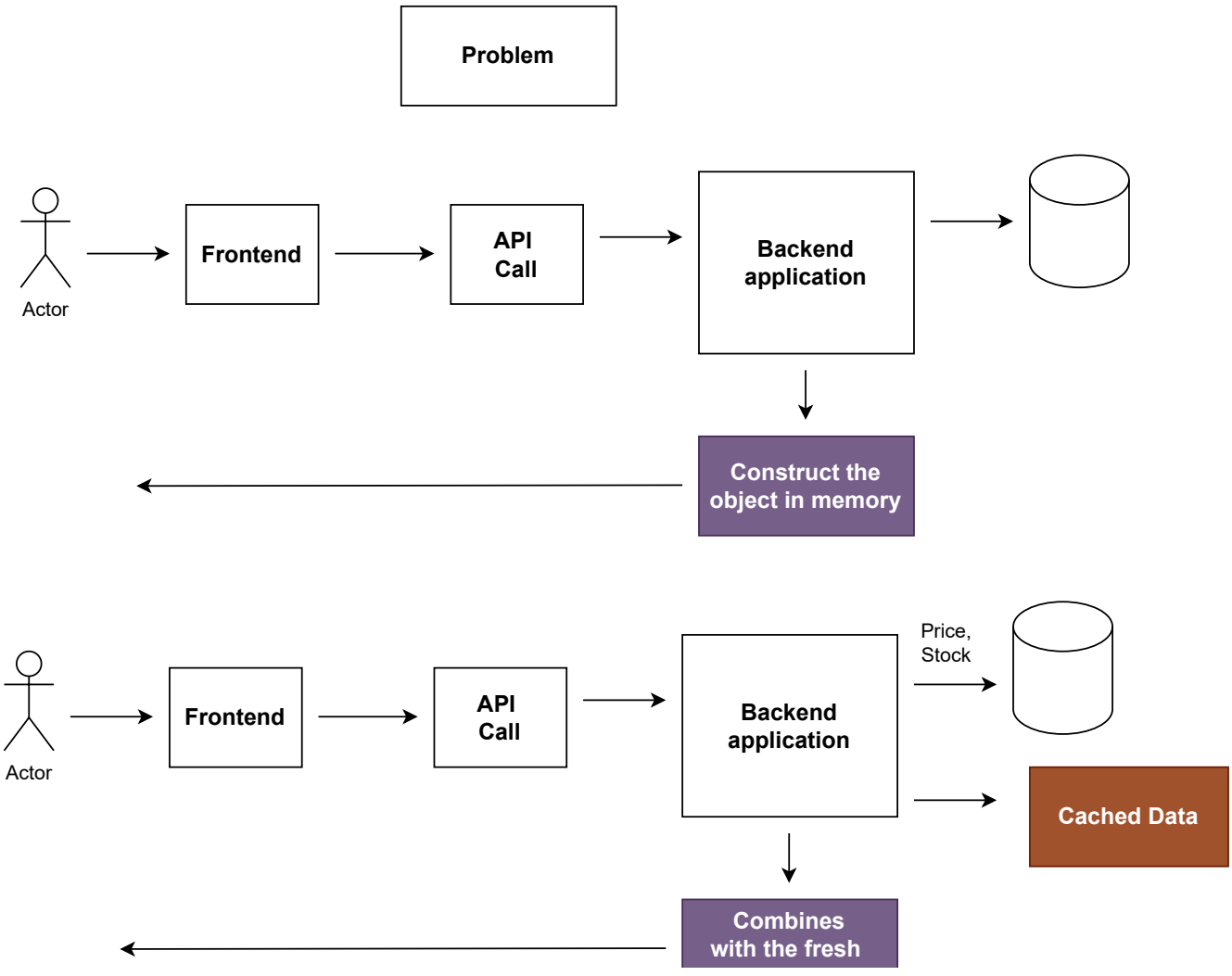

IOT
application
Start AC
Start Light
Start Fan
Start Audio
Authentication
Authentication
Authentication
Authentication
Complex logic
Complex logic
Complex logic
Complex logic
IOT Facade
Actor
Start

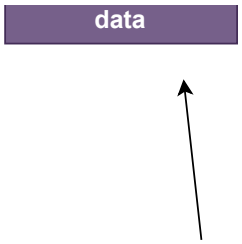

Microservice Architecture
User Service
Order Service
Inventory Service
Payment Service
Business logic
Business logic
Business logic
Business logic
Authentication
Authentication
Authentication
Authentication
API
API
API
API
API Gateway Facade
/dashboard
id
Actor

Flyweight Design Pattern

The Flyweight Pattern is used when you have lots of similar objects, and you want to save memory by reusing the parts that are common to all of them.


Large Scale Ecommerce
Actor
Mobile Phones
Image
Intrinsic
Title
Description
Price
Extrinsic
Stock
Actor
Mobile Phones


Problem
Actor
Frontend
API Call
Backend application
Construct the object in memory
Actor
Frontend
API Call
Backend application
Price, Stock
Cached Data
Combines with the fresh

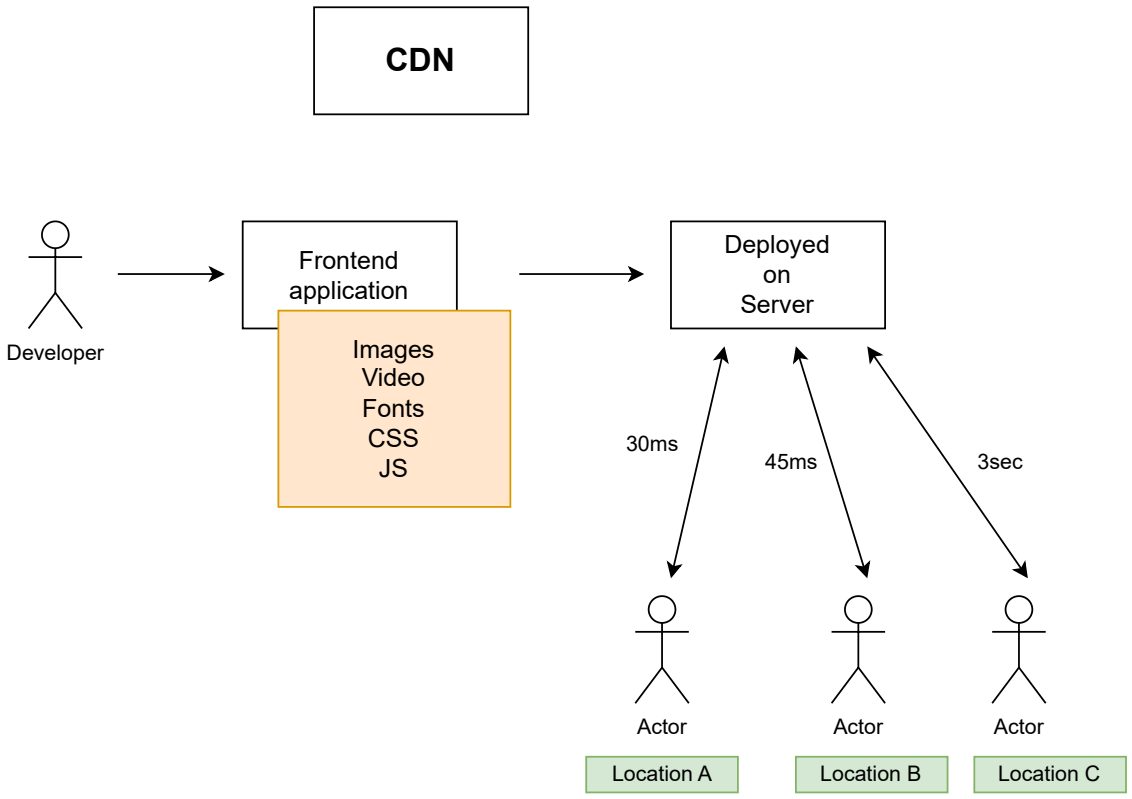


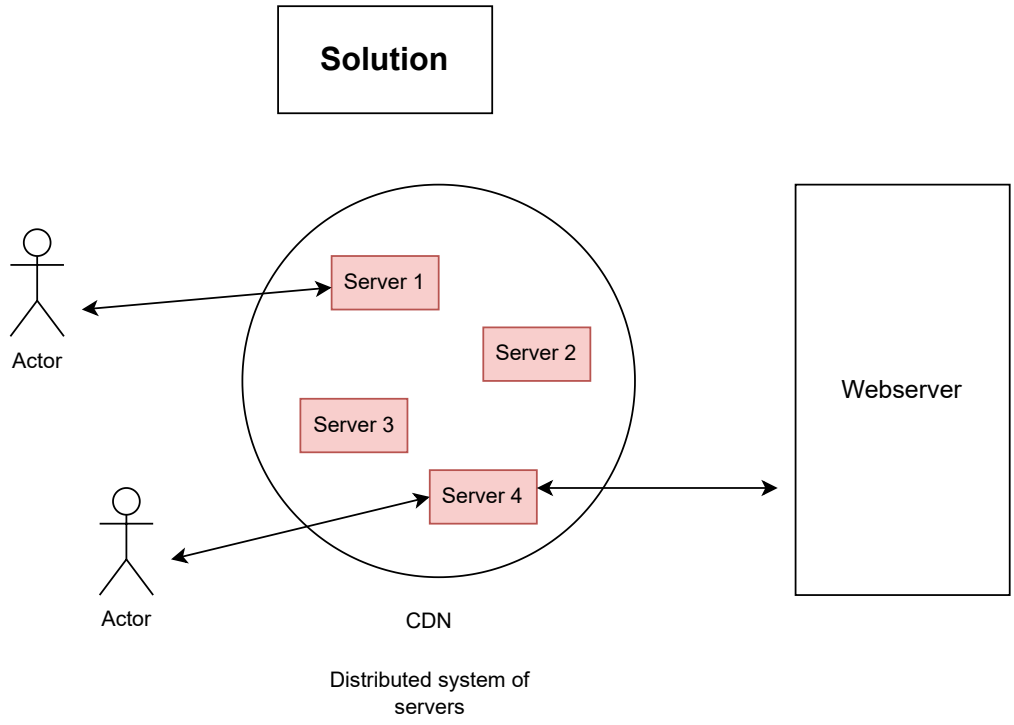
This dramatically reduces the number of duplicated objects in RAM, especially for popular items shown in many places.

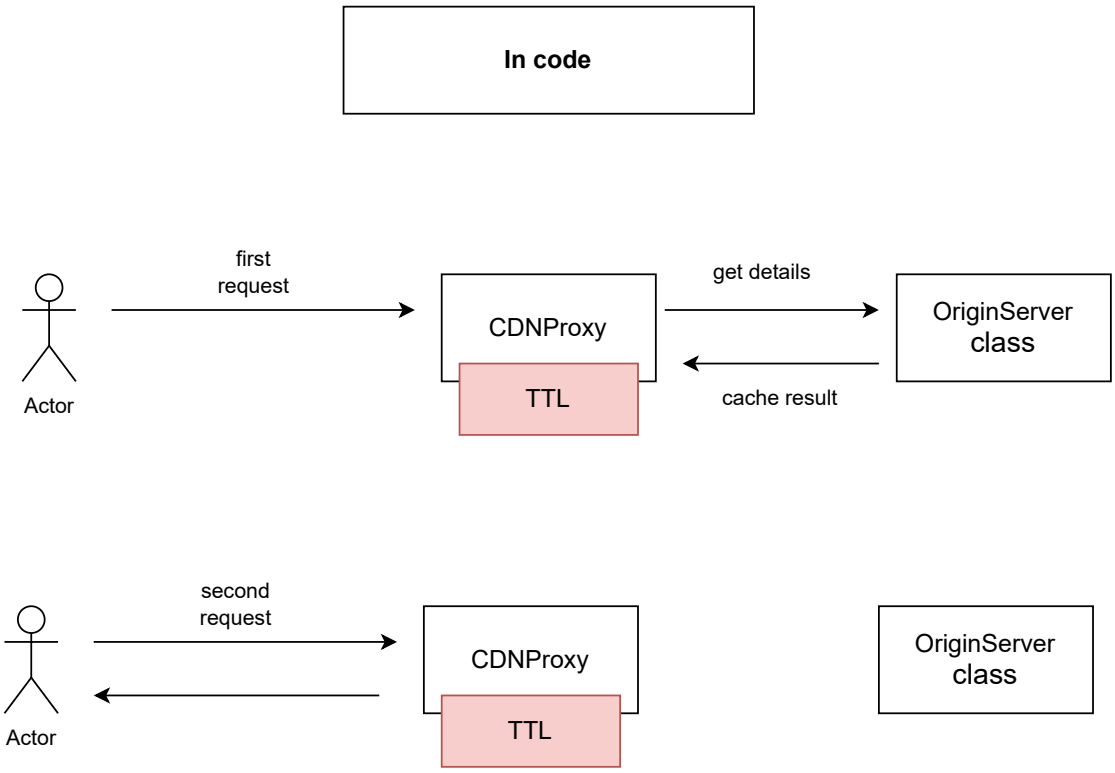
Proxy Design Pattern

A **proxy** is like a smart middleman. It controls access to another object, which might be expensive to create, sensitive, or remote.

Instead of talking to the actual object directly, you talk to the proxy — and the proxy decides when and how to reach the real thing.

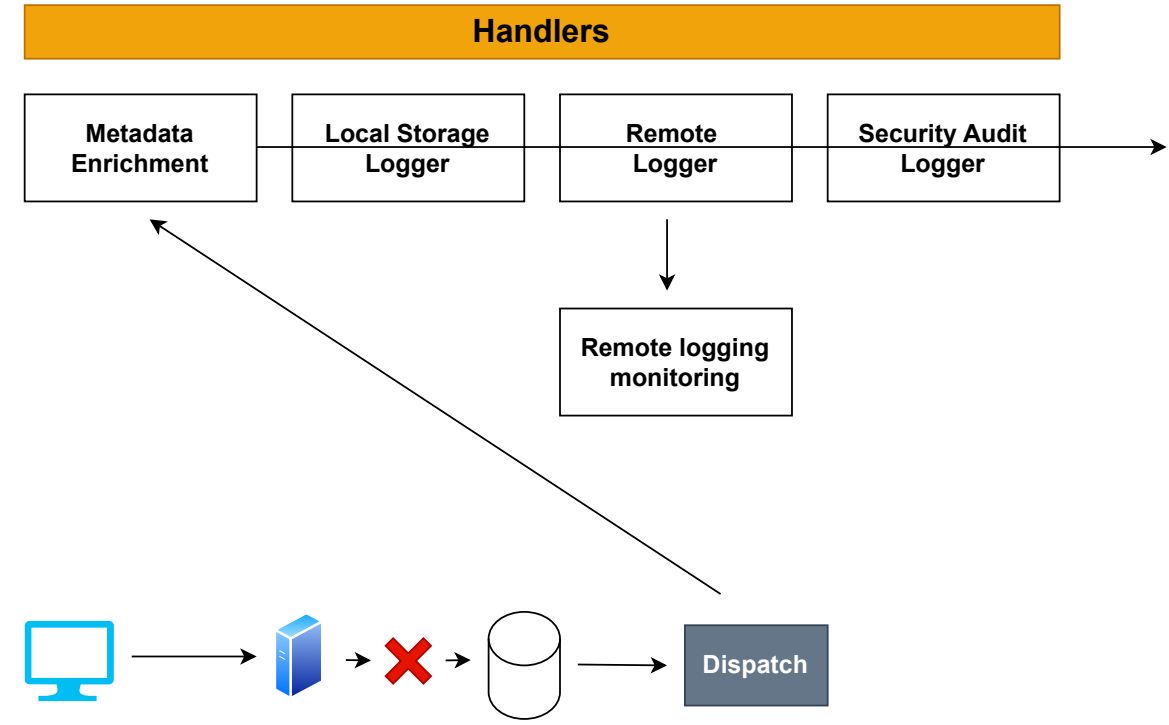

CDN
Developer
Frontend application
Images
Video
Fonts
CSS
JS
Deployed on Server
30ms
45ms
3sec
Actor
Actor
Actor
Location A
Location B
Location C

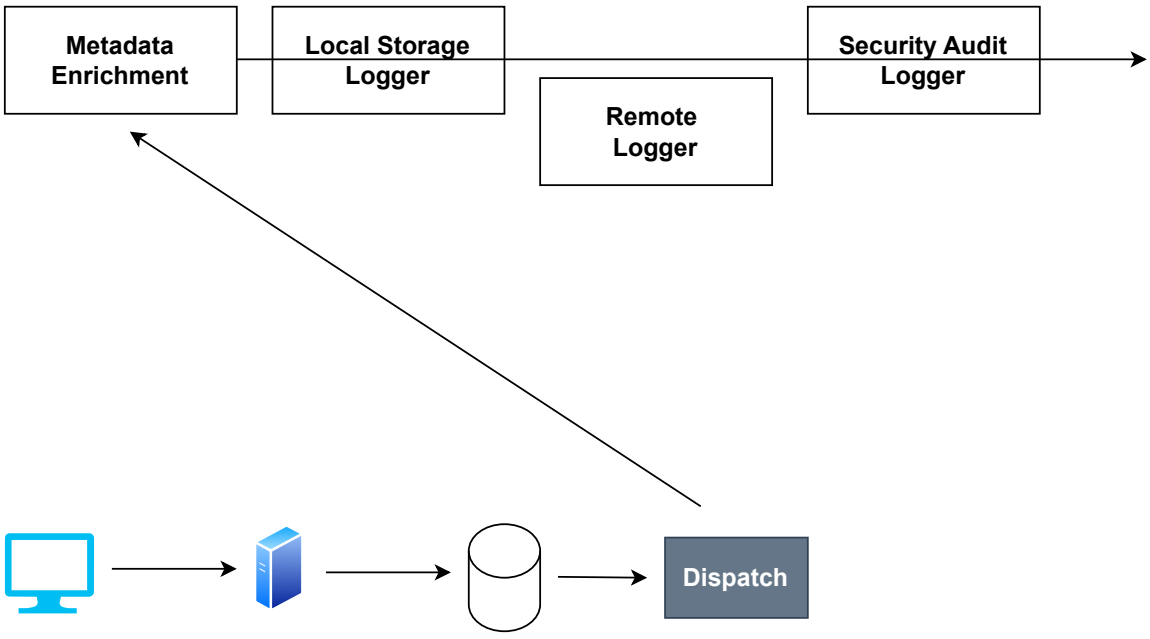

Solution
Actor
Server 1
Server 2
Server 3
Server 4
Actor
CDN
Webserver
Distributed system of servers


In code
first
request
Actor
CDNProxy
TTL
get details
OriginServer
class
cache result
second
request
Actor
CDNProxy
TTL
OriginServer
class

Chain of Responsibility

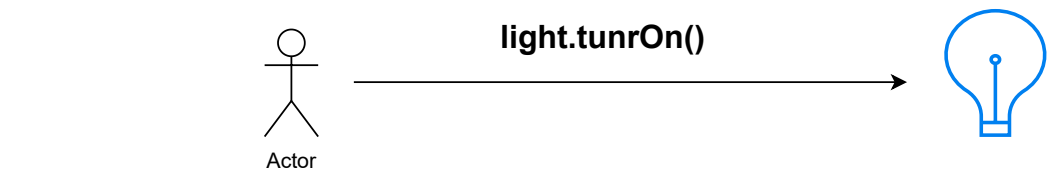
The **Chain of Responsibility** pattern lets you pass a request along a series of handlers, where each one decides either to process the request or pass it to the next in line

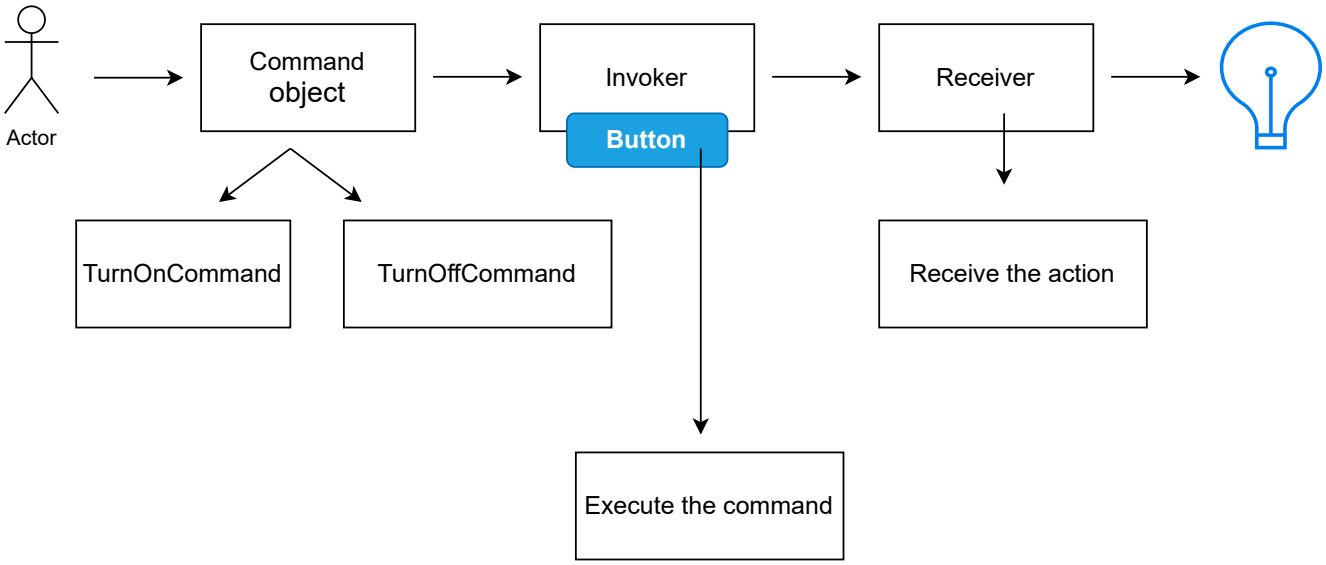

Handlers
Metadata Enrichment
Local Storage Logger
Remote Logger
Security Audit Logger
Remote logging monitoring
Dispatch


Metadata Enrichment
Local Storage Logger
Remote Logger
Security Audit Logger
Dispatch

Command Design Pattern

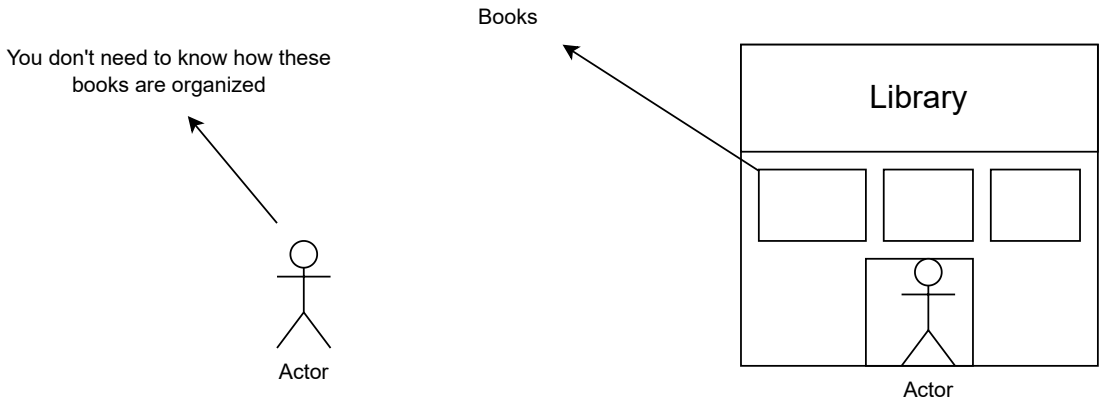
It decouples the sender of a request from the receiver by encapsulating the request as an object.

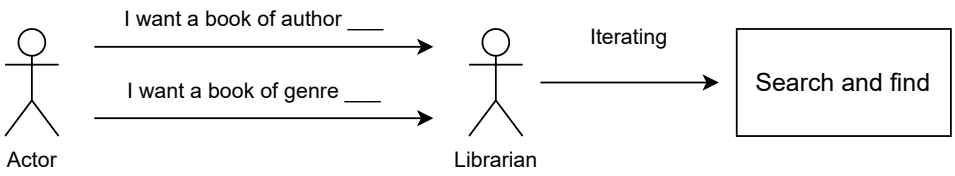

light.tunrOn()
Actor

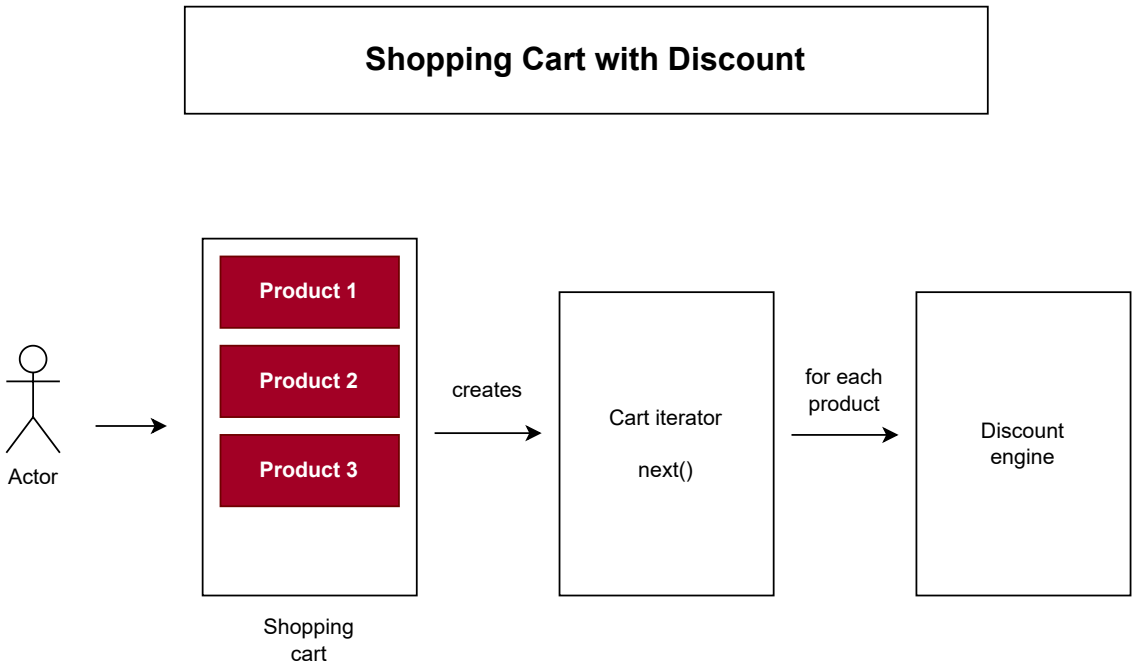

Actor
Command object
Invoker
Button
Receiver
TurnOnCommand
TurnOffCommand
Receive the action
Execute the command

Iterator Design Pattern

The **Iterator Design Pattern** is all about providing a way to **access the elements of a collection sequentially** without exposing its underlying representation


Books
You don't need to know how these books are organized
Library
Actor
Actor

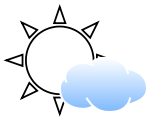

I want a book of author ___
I want a book of genre ___
Iterating
Search and find
Actor
Librarian

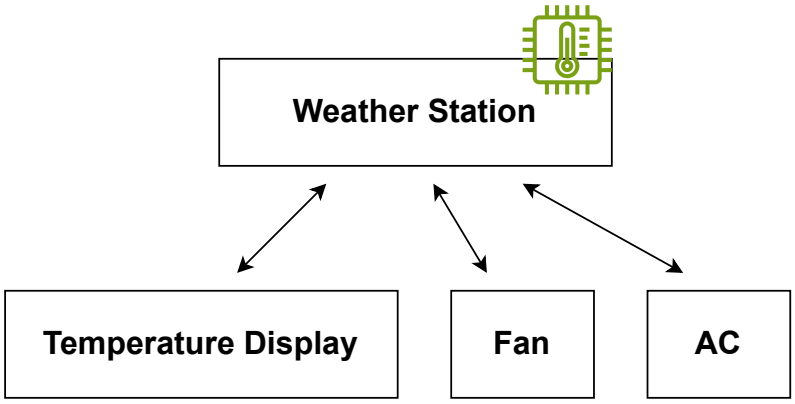

Shopping Cart with Discount
Product 1
Product 2
Product 3
Actor
creates
Cart iterator
next()
for each
product
Discount
engine
Shopping
cart

Observer Design Pattern

The **Observer Pattern** defines a one-to-many relationship between objects. When one object (the **subject**) changes state, all its dependents (the **observers**) are notified and updated automatically.

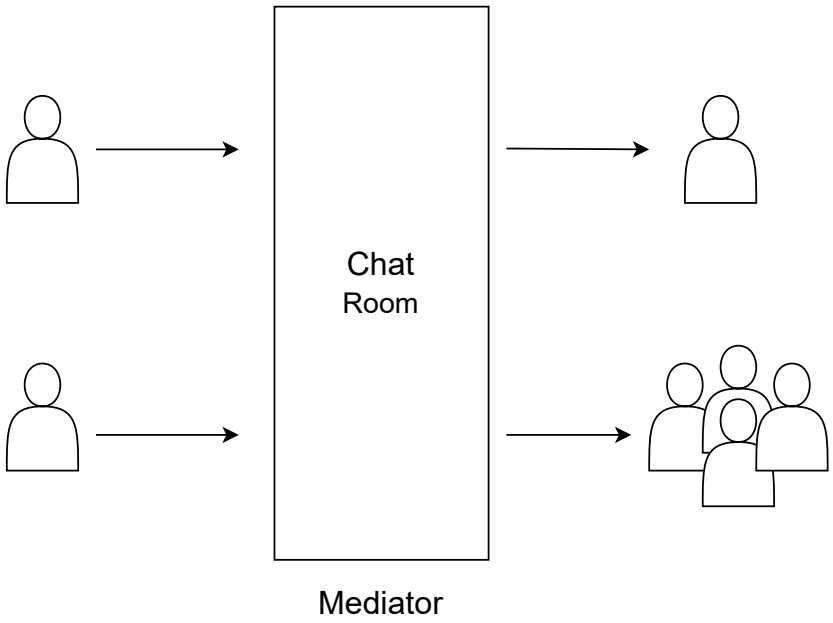
Notification System





Mediator Design Pattern

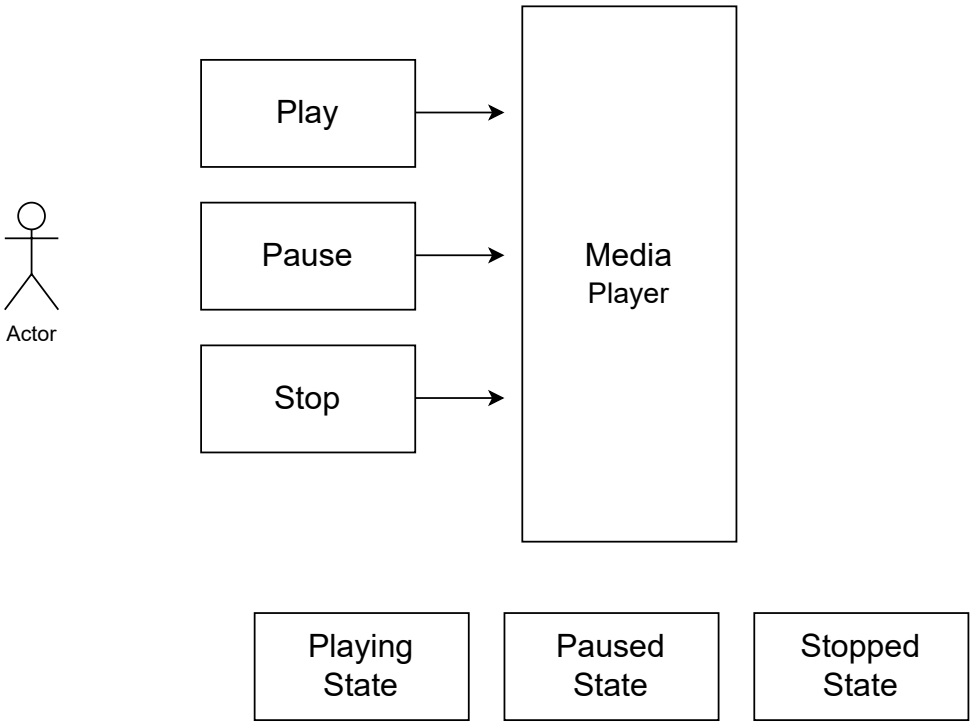
The **Mediator Pattern** centralizes communication between objects, so they don’t talk to each other directly. Instead, they interact through a **mediator**, which coordinates and routes messages.


Chat
Room
Mediator

Feature	Observer Pattern	Mediator Pattern
Purpose	Notify multiple objects when one changes	Centralize complex communication between objects
Communication Style	One-to-many (broadcast)	Many-to-many (orchestrated via mediator)
Coupling	Loose coupling between subject and observers	Loose coupling between colleagues via mediator
Control Flow	Decentralized—each observer reacts independently	Centralized—the mediator controls interactions
Use Case	Event systems, reactive UIs, pub-sub	UI coordination, workflow engines, chat systems
Example Analogy	YouTube channel notifying subscribers	Air traffic control managing planes

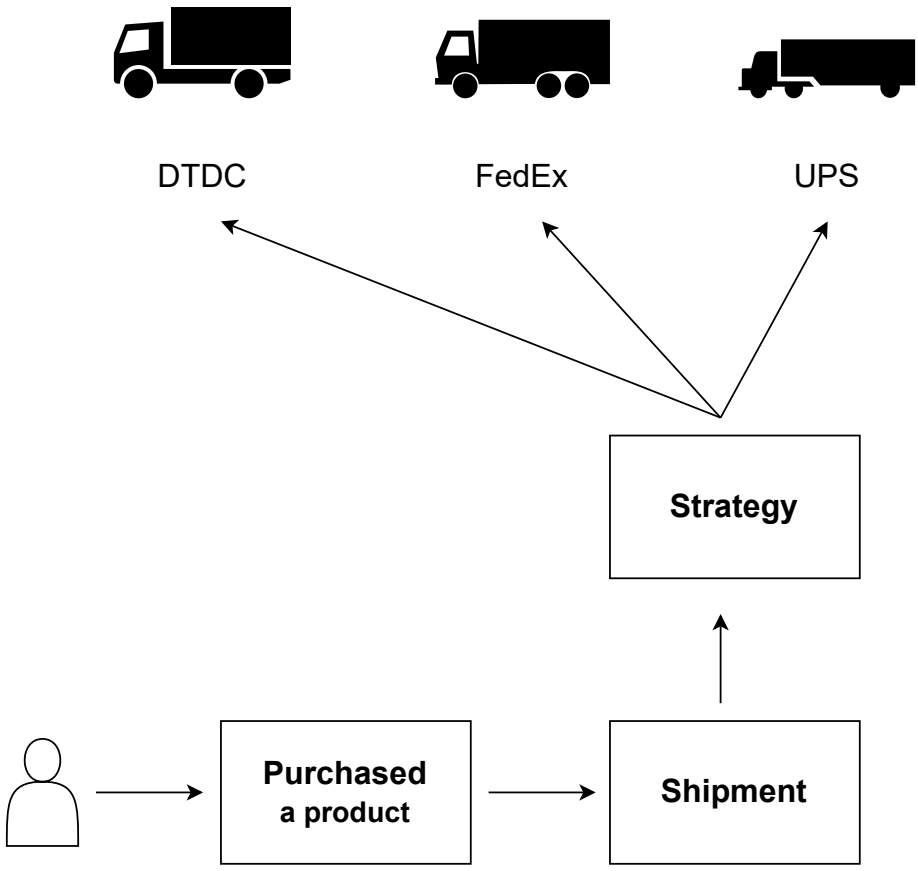
State Design Pattern

The State Pattern allows an object to change its behavior when its internal state changes—as if the object changed its class. Instead of using conditionals, you delegate behavior to state-specific classes.


Actor
Play
Pause
Stop
Media
Player
Playing
State
Paused
State
Stopped
State

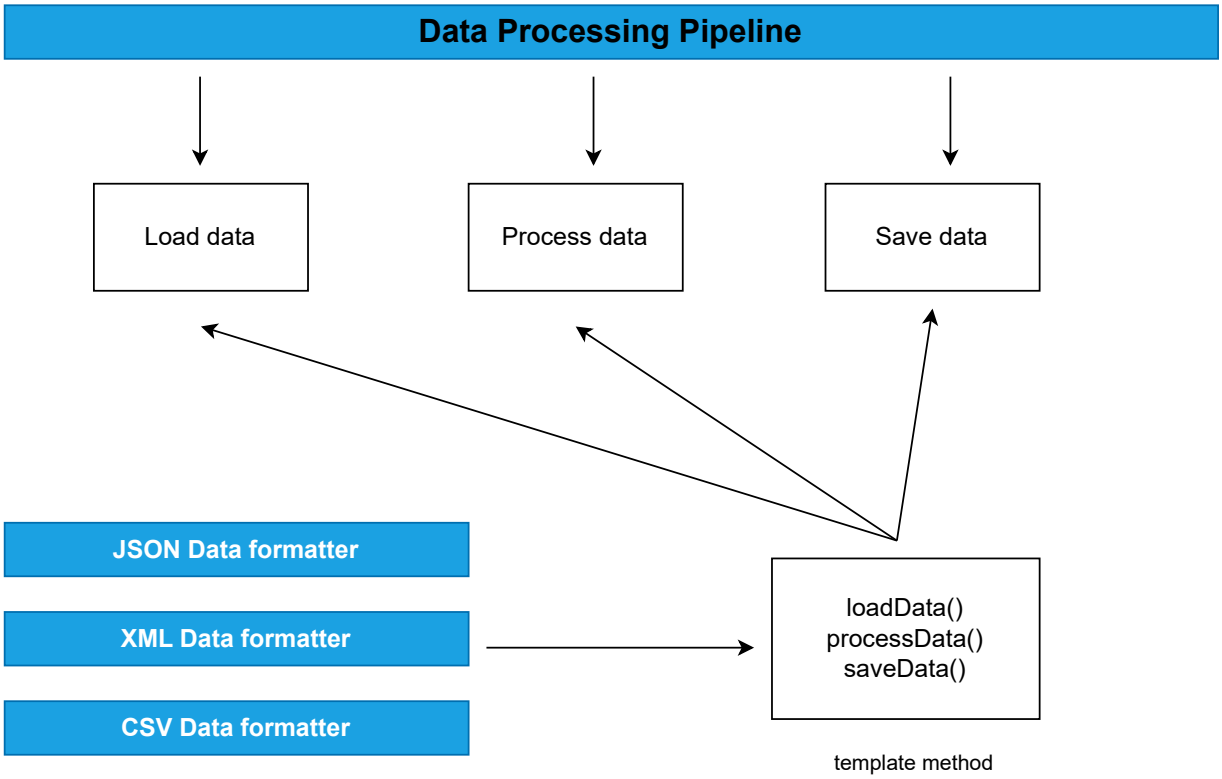
Strategy Design Pattern

The **Strategy Pattern** defines a family of algorithms (or behaviors), encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from the clients that use it.


DTDC
FedEx
UPS
Strategy
Purchased
a product
Shipment

Template Method Design Pattern

The **Template Method Pattern** is a behavioral design pattern that defines the **skeleton of an algorithm** in a base class method, deferring some steps to subclasses. It allows subclasses to redefine specific steps of the algorithm without changing its overall structure.


Data Processing Pipeline
Load data
Process data
Save data
JSON Data formatter
XML Data formatter
CSV Data formatter
loadData()
processData()
saveData()
template method

Thank You